



Configuring AWS EKS

Understanding Kubernetes for Orchestration

Introduction

Once an **Amazon Elastic Kubernetes Service (EKS) cluster** is created, it must be **configured properly** to deploy and manage applications efficiently.

Configuration includes:

- **Connecting to the EKS cluster**
- **Managing authentication and access**
- **Setting up networking and security**
- **Deploying applications and monitoring the cluster**

This guide will cover:

- **Why configure EKS?**
 - **How to connect to an EKS cluster**
 - **Configuring IAM roles and networking**
 - **Deploying and testing applications**
 - **Common issues and FAQs**
-

Why Configure AWS EKS?

After creating an EKS cluster, proper configuration is essential to:

- **Enable secure access** – Authenticate users and applications securely.
- **Deploy workloads** – Ensure smooth application deployment.
- **Monitor and manage resources** – Optimize cluster performance.
- **Integrate with AWS services** – Use AWS IAM, CloudWatch, and Load Balancers.



💡 Without proper configuration, an EKS cluster may not function efficiently or securely.

Step 1: Connect to an EKS Cluster

Before configuring the cluster, ensure that the following tools are installed:

- **AWS CLI** (Command Line Interface)
- **kubectl** (Kubernetes CLI)
- **eksctl** (EKS Management Tool)

Check if the cluster exists:

```
eksctl get cluster --region ap-south-1
```

Retrieve cluster details:

```
aws eks describe-cluster --name my-cluster --region ap-south-1
```

Configure **kubectl** to Use the EKS Cluster

Update **kubectl** with the cluster configuration:

```
aws eks update-kubeconfig --region ap-south-1 --name my-cluster
```

Verify the connection:

```
kubectl get nodes
```

💡 If you see nodes listed, your connection is successful!

Step 2: Configure IAM Roles and Access Control

AWS IAM (Identity and Access Management) controls who can access the EKS cluster.



Add an IAM User to the EKS Cluster

1. Retrieve the **current configuration**:

```
kubectl get configmap aws-auth -n kube-system -o yaml
```

2. Edit the **aws-auth** ConfigMap to add a new IAM user:

```
kubectl edit configmap aws-auth -n kube-system
```

3. Add the following under **mapUsers**:

```
mapUsers: |
- userarn: arn:aws:iam::123456789012:user/developer
  username: developer
  groups:
    - system:masters
```

4. Save and apply the changes.

Verify user access:

```
kubectl get nodes --as developer
```



*IAM roles ensure **secure user access** without sharing AWS credentials directly.*

Step 3: Configure Networking in EKS

Enable Cluster Communication with VPC

EKS requires a **VPC (Virtual Private Cloud)** for networking. To find the VPC ID:



```
aws eks describe-cluster --name my-cluster --query  
"cluster.resourcesVpcConfig.vpcId"
```

Ensure the required security groups are attached:

```
aws ec2 describe-security-groups --filters Name=vpc-id,Values=<VPC-ID>
```

Allow Public and Private Access

To allow both public and private access to the cluster:

```
aws eks update-cluster-config --name my-cluster --resources-vpc-config  
endpointPublicAccess=true,endpointPrivateAccess=true
```

💡 *Configuring networking properly ensures secure and **high-performance communication** between services.*

Step 4: Deploy a Sample Application on EKS

Create a Deployment

1. Create a file named **deployment.yaml**:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-deployment  
spec:  
  replicas: 2  
  selector:  
    matchLabels:  
      app: nginx
```



```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:latest
        ports:
          - containerPort: 80
```

2. Deploy the application:

```
kubectl apply -f deployment.yaml
```

3. Verify that the Pods are running:

```
kubectl get pods
```



*EKS Deployments ensure that applications run **with high availability and scalability**.*

Step 5: Expose the Application Using a Load Balancer

To make the application accessible, create a **LoadBalancer Service**:

1. Create a file named **service.yaml**:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
```



```
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

2. Apply the Service:

```
kubectl apply -f service.yaml
```

3. Retrieve the external access URL:

```
kubectl get services
```

💡 AWS automatically assigns a **public IP address** via the Load Balancer.

Step 6: Enable Auto Scaling in EKS

EKS supports **Pod scaling** and **Node scaling**.

Enable Horizontal Pod Autoscaler (HPA)

1. Install the metrics server:

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/co
mponents.yaml
```

2. Enable HPA for the Nginx deployment:



```
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=2  
--max=5
```

3. Check the status of HPA:

```
kubectl get hpa
```

💡 HPA adjusts the number of Pods based on CPU usage.

Common Issues and Troubleshooting

Problem	Solution
Cannot connect to EKS cluster	Ensure <code>aws eks update-kubeconfig</code> was run
Pods stuck in <code>Pending</code>	Check node availability using <code>kubectl get nodes</code>
Load Balancer not getting an external IP	Verify <code>type: LoadBalancer</code> is set in Service YAML
<code>Access denied</code> when managing EKS	Ensure IAM user has necessary policies attached

Know More (FAQs)

1. What is the difference between EKS and self-managed Kubernetes?

- **EKS** is fully managed by AWS.
- **Self-managed Kubernetes** requires users to configure the control plane manually.



2. How do I monitor my EKS cluster?

Use **Amazon CloudWatch**, **Prometheus**, or **Grafana** for logging and monitoring.

3. Can I run EKS without EC2 nodes?

Yes, by using **AWS Fargate**, which runs Pods **without needing virtual machines**.

4. How do I restrict access to my EKS cluster?

Use **IAM roles** and restrict permissions using **aws-auth** ConfigMap.

5. How do I delete an EKS cluster?

To delete an EKS cluster, run:

```
eksctl delete cluster --name my-cluster
```

Conclusion

Properly **configuring AWS EKS** ensures that your Kubernetes applications **run securely, scale efficiently, and integrate seamlessly with AWS services**.

👉 Try setting up an **EKS cluster**, **deploying an application**, and **scaling it using HPA!**